

JC2001 - Introduction to Software Engineering

Group Project - Project Proposal

Academic year 2026-27

Design and Implementation of an On-Campus Second-Hand Trading Platform for University Students

Campus Market - a campus-only marketplace for buying, selling and swapping second-hand goods between students

Group number: 9

Programme of study: BSc (Artificial Intelligence)

Group member	Name	Student ID
Group member 1	<Full name>	<Student ID>
Group member 2	<Full name>	<Student ID>
Group member 3	<Full name>	<Student ID>
Group member 4	<Full name>	<Student ID>
Group member 5	<Full name>	<Student ID>

Submitted to the MyAberdeen course assessment area · Submission date: <date>

Project Title

Design and Implementation of an On-Campus Second-Hand Trading Platform for University Students (working name: Campus Market). Campus Market is a web application that lets students at a single university list, discover, reserve and hand over second-hand goods to one another. It is deliberately narrow: it serves one campus, it carries no payment, and it exists to make an already-happening behaviour - students passing textbooks, electronics and everyday items to each other - safer, faster and easier to find.

Problem

Every academic year a large volume of usable goods changes hands between students. Textbooks bought for a single semester, laptops replaced on an upgrade, bicycles, calculators, sports equipment and kitchen items all sit unused in dormitory rooms after their owners stop needing them, while the demand for those exact items exists a few hundred metres away in the next year group.

That demand is currently served by informal channels, and each fails in a different way. General-purpose marketplaces are built for strangers rather than for a campus: listings are drowned in results from the wider city, handover has to be negotiated with someone who may be an hour away, and the platform takes a commission, so a student selling a 40-yuan textbook has no incentive to use one. Social media groups are where most transactions actually happen, but a chat stream is a poor catalogue - a listing is buried within hours, and there is no search, no filter and no way to tell whether an item is still available. Noticeboards reach only the people who walk past.

Two problems are specific to this setting and explain why a general-purpose platform cannot simply be pointed at a campus. The first is **trust**: students transact with people they may share a lecture theatre with but do not know, so they need to know that the other party belongs to the same institution, and need a public place to meet. The second is **state**: informally there is no authoritative answer to "is this still available?", so sellers answer it dozens of times, buyers waste journeys on items that have already gone, and the same textbook is re-posted repeatedly by different people.

The consequence is a market that is inefficient in both directions: sellers abandon goods they cannot shift or give them away, and buyers pay full price for new items because they never saw the used alternative. Solving this is worthwhile on three counts: it saves money for

students, who are among the most price-sensitive consumers in the population; it reduces waste, because every item reused is an item not manufactured again and not discarded; and it is practically useful to the institution, since a working campus marketplace reduces the volume of usable goods abandoned at the end of the academic year.

Solution

We propose to design and build **Campus Market**, a lightweight web application that provides a campus-scoped catalogue of second-hand goods, a structured way to negotiate a handover, and an explicit, server-enforced record of where each item stands in the transaction. The solution has five parts.

- **A campus-scoped account.** Students register with a username, password and student identifier, so every listing is attributable to a member of the institution. The anonymity that makes general-purpose marketplaces risky is removed without collecting more personal data than a transaction needs.
- **A searchable, filtered catalogue.** Sellers publish a listing with a title, description, price, category, condition and up to three photographs. Buyers search by keyword across title and description, filter by category, and sort by recency or price, with results paginated so the catalogue stays usable as it grows.
- **A public comment thread on every listing,** so a buyer can ask publicly whether an item is available and every later buyer can read the answer - which removes the most common source of wasted messages.
- **A server-enforced transaction state.** Every listing occupies exactly one of four states - on sale, reserved, sold, or taken offline - and moves between them only through actions the server validates against the caller's role and the item's current state. A buyer reserves an item; the seller either confirms the sale or cancels the reservation.
- **Direct messaging and a personal centre.** Buyers and sellers exchange private messages about a specific item, and every student has a personal centre showing their own listings, saved items, sold items and conversations.

The scope is intentionally narrow. The platform carries **no online payment**; money changes hands in person at handover, exactly as it does today. This is a design decision rather than a missing feature - taking payment would bring financial regulation, dispute resolution and fraud handling that are far beyond a course project, and excluding it excludes them entirely.

What the software contributes is everything around the money - discovery, availability, contact and record - which is precisely the part that is currently broken.

A first version has already been built as a proof of concept and is described in the accompanying technical report: a Python Flask application with a SQLite database and a framework-free single-page front end, running on a lecture-room laptop with one command. Twenty-three routes implement the features above and eighty automated test cases exercise them, all of which pass. The remaining work is the documentation, presentation and evaluation the module requires, together with the improvements listed under Limitations in the technical report.

Objectives

We intend to achieve the following specific objectives. Each is written so that its completion can be demonstrated by an artefact rather than asserted.

- (1) **To specify the requirements** of a campus second-hand trading platform by identifying its users and stakeholders, and producing a prioritised set of functional and non-functional requirements that the rest of the project can be traced against.
- (2) **To design the system architecture and data model**, expressed as a layered architecture, a domain model, a relational schema and a set of interaction models, with the key decisions and their trade-offs recorded.
- (3) **To implement a working proof of concept** demonstrating the core features end to end: registration and session management, listing creation with image upload, catalogue browsing with search, filtering, sorting and pagination, item detail with comments and favourites, private messaging, and server-validated transaction status management.
- (4) **To verify the proof of concept against the stated requirements** through automated unit, component and interface testing, negative and boundary testing, performance measurement, and an evaluation of the interface on desktop and mobile viewports - reporting the results honestly, including cases that fail or remain unverified.
- (5) **To deliver the documentation and presentation** required by the module: a technical report, a user manual, presentation slides and a recorded demonstration, all consistent with the delivered software.

Benefits

Students as sellers list an item once and no longer repost it into a chat stream that buries it within hours; the listing stays discoverable until it is sold, and the status field stops enquiries about items that have already gone, so goods abandoned at the end of the year recover some of their value. **Students as buyers** get a searchable catalogue of goods that are physically reachable, with photographs, condition ratings and a price; because every seller belongs to the same institution and handover happens on campus, a transaction is low-risk and can be completed between lectures, and the public comment thread answers the availability question before a journey is made.

The student community and the institution benefit because reuse displaces new manufacture and reduces the waste generated at the end of each academic year. A campus marketplace costs nothing to operate and requires no payment infrastructure, no commercial partnership and no handling of money or personal financial data. Finally, **the project team** gains a realistic, self-contained exercise in the full software engineering life cycle - eliciting requirements from real users, modelling a state machine whose correctness matters, choosing and defending an architecture, implementing and testing it, and reporting results honestly, including the parts that fell short.

Three properties make these benefits achievable rather than aspirational. The application is **cheap to run**, needing no server licence and no payment provider. It is **easy to adopt**, because it asks students to do nothing new - the behaviour it formalises already happens. And it is **safe by default**, because the scope excludes money and the only personal data held is a display name, an optional email address and an optional student identifier.

Timeline

Table 1 gives the estimated schedule, aligned to the four project-update checkpoints. Phase 1 is reported as complete because the team built an early prototype in order to validate the scope before committing to a written specification.

Table 1.1 *Estimated project timeline*

Phase	Main activities	End date
Phase 1 - Scope and prototype	Problem definition, stakeholder identification, supervisor discussion, and a first end-to-end prototype to test whether the scope was feasible	October 2026

Phase	Main activities	End date
Phase 2 - Requirements and design	Elicitation with student users, functional and non-functional requirements, use case model, architecture, data model, interaction models; project update 1	November 2026
Phase 3 - Implementation	Completion of the feature set, front-end rework, interface localisation to English, seed data for demonstration; project update 2	December 2026
Phase 4 - Verification and evaluation	Automated test suite, negative and boundary testing, performance and scaling measurement, interface evaluation, defect fixing; project update 3	January 2027
Phase 5 - Documentation and presentation	Technical report, user manual, slide deck, recorded demonstration and workload profile; project update 4	January 2027

Action Plan

Table 2 lists the activities required to achieve each objective, the member responsible, the deadline and the current status. Member assignments are roles that map onto the named members on the title page.

Table 1.2 *Action plan against the five objectives*

Obj.	Action	Assigned to	Deadline	Status
1	Interview student users and record the current workarounds used to buy and sell goods on campus	Member 1	Oct 2026	Complete
1	Agree scope and boundaries with the academic supervisor	Member 2	Oct 2026	Complete
1	Derive and prioritise the functional requirements	Members 1, 3	Nov 2026	Complete
1	Derive the non-functional requirements covering usability, performance, reliability, security and maintainability	Member 3	Nov 2026	Complete
1	Build the use case model and write the detailed use case descriptions	Members 2, 4	Nov 2026	Complete
2	Select an architectural style and justify it against at least two alternatives	Member 2	Nov 2026	Complete
2	Design and normalise the relational schema, and define the REST interface, response envelope and error model	Members 4, 5	Nov 2026	Complete
2	Produce the interaction models for publishing, transacting and messaging	Member 4	Dec 2026	Complete
2	Design the user interface including empty, loading and error states	Member 5	Dec 2026	Complete
3	Implement accounts, sessions, listing creation, editing and soft deletion	Member 2	Dec 2026	Complete
3	Implement the catalogue query with search, filter, sort and pagination	Member 1	Dec 2026	Complete

Obj.	Action	Assigned to	Deadline	Status
3	Implement image upload with an extension allow-list, size cap and generated filenames	Member 3	Dec 2026	Complete
3	Implement the transaction state machine with role and state guards	Member 4	Dec 2026	Complete
3	Implement favourites, comments and private messaging	Member 3	Dec 2026	Complete
3	Rework the front end into a single-page application with all views, feedback states and responsive breakpoints	Member 5	Jan 2027	Complete
4	Write the automated test suite covering every JSON endpoint, plus negative and boundary cases for validation, authorisation and the transition guards	Members 1, 4	Jan 2027	Complete
4	Measure response time per operation and the scaling of catalogue reads	Member 3	Jan 2027	Complete
4	Evaluate the interface on desktop and mobile and check every screen for console errors	Member 5	Jan 2027	Complete
4	Produce the requirements traceability matrix and record unresolved gaps	Member 2	Jan 2027	In progress
5	Write the technical report and the user manual, keeping both consistent with the delivered software	Members 2, 3	Jan 2027	In progress
5	Assemble the slide deck and record the fifteen-minute presentation	All members	Jan 2027	Planned
5	Complete and sign the group workload profile	All members	Jan 2027	Planned

Relationship to the other submissions

This proposal is written to be read alongside the other two assessed submissions. The technical report takes the objectives above and reports against them, including the requirements that remain unmet; the proof-of-concept software is the artefact it evaluates. Where this proposal claims that something has been achieved, the technical report supplies the evidence and the submitted source code provides the means to reproduce it.